

Analyse détaillée du fichier script.js

Ce script gère toute l'intelligence de l'application : interroger le serveur météo, mettre à jour la carte, gérer le cache pour économiser la batterie et les données, et assurer que l'interface reste réactive.

0. Le Démarrage Sécurisé (DOMContentLoaded)

```
document.addEventListener('DOMContentLoaded', () => { ... });
```

- **Le Concept** : On enveloppe **tout** le code dans cet événement.
- **Pourquoi ?** On attend que le HTML soit totalement chargé et "lu" par le navigateur avant d'essayer de manipuler les éléments. Si on ne fait pas ça, le script pourrait chercher un bouton qui n'existe pas encore.

1. Les Outils "Helpers" (Section 0)

Au lieu d'écrire du code répétitif partout, vous avez créé votre propre petite "boîte à outils".

La fonction el (Element Builder)

C'est une alternative sécurisée à innerHTML.

```
function el(tag, props = {}, children = []) {  
  const node = document.createElement(tag); // Crée une balise (ex: <div>)  
  Object.assign(node, props); // Lui ajoute des attributs (ex: class="box")  
  // ... ajoute les enfants (texte ou autres balises)  
  return node;  
}
```

- **Architecture** : C'est une mini-version de ce que fait React. Cela permet de construire l'interface (les cartes météo) proprement en JavaScript sans manipuler des chaînes de caractères risquées (protection contre les failles XSS).

La fonction setAriaBusy

```
function setAriaBusy(isBusy) {
```

```
resultDiv.setAttribute('aria-busy', String(isBusy));
}
```

- **Accessibilité (A11Y)** : Dit aux lecteurs d'écran pour aveugles : *"Attention, je suis en train de charger quelque chose, attends un peu"*.

2. Le Menu Mobile (Section 1)

Ce bloc gère l'ouverture/fermeture du menu sur téléphone.

```
mobileMenu.setAttribute('aria-expanded', String(isOpen));
```

- **Pourquoi cette ligne ?** En plus de changer l'aspect visuel (CSS), on modifie un attribut sémantique pour dire aux technologies d'assistance si le menu est "Ouvert" ou "Fermé".
- **keydown 'Escape'** : Permet de fermer le menu avec la touche "Échap" du clavier. C'est un standard de navigation web.

3. Gestion de l'État et Configuration (Section 2)

C'est la "mémoire" locale de votre application.

```
const state = {
  requestSeq: 0, // Compteur de requêtes (pour éviter les mélanges)
  activeController: null // Pour annuler une requête en cours
};
```

```
const CONFIG = {
  TIMEOUT_MS: 12000, // On abandonne si > 12s
  CACHE_TTL_MS: 2 * 60 * 1000, // On garde les données en mémoire 2 minutes
  // ...
};
```

- **Architecture** : Centraliser les "chiffres magiques" (12000, 2 minutes) dans CONFIG permet de régler le comportement de l'app facilement sans fouiller dans le code complexe.

Le Système de Cache (getCache, setCache)

```
function getCache(q) {
  // ... Vérifie si on a déjà la donnée et si elle n'est pas périmée
}
```

- **Utilité** : Si l'utilisateur tape "Paris", puis "Lyon", puis refait "Paris", l'app ne recontactera pas le serveur pour Paris. Elle affichera le résultat instantanément. C'est excellent pour la performance.

4. La Communication Réseau "Senior" (Section 3)

C'est la partie la plus impressionnante techniquement.

fetchJsonWithTimeout

```
const controller = new AbortController();
// ...
const abortOnTimeout = () => controller.abort(...);
```

- **Le problème résolu** : La fonction fetch de base attend indéfiniment. Ici, on force l'arrêt si le serveur est trop lent (Timeout), pour ne pas bloquer l'interface.

fetchWeatherSenior (L'Algorithme principal)

Cette fonction gère la complexité du réseau.

1. **Vérification du Cache** : if (cached) return ... (Si on a l'info, on s'arrête là).
2. **Annulation des requêtes précédentes** :
if (state.activeController) state.activeController.abort();
 - *Scénario* : Je tape "Par", l'app cherche. Je tape vite "i", puis "s" (Paris). L'app annule la recherche pour "Par" et "Pari" pour ne chercher que "Paris". Ça économise la bande passante.
3. **Gestion des "Race Conditions" (Conditions de course)** :
const seq = ++state.requestSeq;
// ... plus tard ...
if (seq !== state.requestSeq) return { ok: false, ignored: true };
 - *Explication* : Si la réponse pour "Tokyo" (demandé il y a 5s) arrive APRES la réponse pour "Paris" (demandé il y a 1s), sans ce code, l'interface afficherait Tokyo alors que l'utilisateur veut voir Paris. Ce compteur seq assure qu'on affiche toujours la *dernière* demande valide.

4. Mécanisme de Retry (Réessai) :

```
await sleep(CONFIG.RETRY_BASE_DELAY_MS * Math.pow(2, attempt));
```

- *Backoff Exponentiel* : Si le serveur plante, on réessaie après 0.5s, puis 1s, puis 2s. C'est une technique d'ingénierie réseau pour ne pas surcharger un serveur en difficulté.

5. Rendu de l'Interface (renderWeather) (Section 4)

Cette fonction prend les données brutes (JSON) et fabrique le HTML.

```
const weatherIcons = { "01d": "☀️", "02d": "☁️", ... };
```

- **Mapping** : Transforme les codes obscurs de l'API météo (ex: "01d") en emojis compréhensibles par les humains.

// Création dynamique des cartes

```
const cards = [ { emoji: '🌡️', label: 'Température', ... }, ... ];
```

```
for (const c of cards) { ... }
```

- **Code Propre** : Au lieu de copier-coller 6 fois le code HTML d'une carte, on fait une boucle sur un tableau de données. Si on veut changer le design des cartes, on le change à un seul endroit.

6. La Carte Leaflet (Section 5)

```
const map = L.map('map').setView(...)
```

```
L.tileLayer(...).addTo(map);
```

- Initialise la librairie Leaflet (chargée dans le HTML).
- `map.flyTo(...)` : Crée cette animation fluide de "vol" d'une ville à l'autre au lieu d'une téléportation brutale.

7. Expérience Utilisateur (UX) (Section 6)

Persistance (localStorage & Hash)

```
function saveLastCity(city) { localStorage.setItem(STORAGE_KEY, city); }
```

```
// ...
```

```
history.replaceState(null, '', `#city=${value}`);
```

- **LocalStorage** : Si l'utilisateur ferme l'onglet et revient demain, l'app se souviendra qu'il

regardait "Paris".

- **URL Hash** : Modifie l'URL pour devenir `monsite.com/#city=Paris`. Cela permet à l'utilisateur de copier le lien et de l'envoyer à un ami, qui verra directement la météo de Paris.

8. L'Autocomplétion "Debounced" (Section 7)

```
clearTimeout(debounceTimer);  
debounceTimer = setTimeout(async () => { ... }, 300);
```

- **Le concept du "Debounce" (Anti-rebond) :**
 - *Sans Debounce* : L'utilisateur tape "Marseille". Le code envoie 9 requêtes (M, Ma, Mar, Mars...). C'est lent et ça surcharge le serveur.
 - *Avec Debounce* : Le code attend que l'utilisateur arrête de taper pendant 300ms avant d'envoyer la requête. Il n'envoie qu'une seule requête : "Marseille".

Résumé Architectural

Ce fichier `script.js` transforme une page web statique en une **Application Web Progressive (PWA)** robuste.

1. **Fiabilité** : Elle gère les réseaux instables (Retries, Timeouts).
2. **Performance** : Elle ne gaspille pas de ressources (Cache, Debounce, Annulation de requêtes).
3. **Utilisabilité** : Elle se souvient de l'utilisateur (LocalStorage) et permet le partage (Hash URL).
4. **Maintenabilité** : Le code est découpé en petites fonctions logiques (`el`, `renderWeather`, `fetchWeatherSenior`).